



Angular – 21.X.X

Productivity Meets Scalability



8. Write the Unit Test File (math.test.js)
9. Jest automatically scans for files ending in .test.js or .spec.js.

```
// math.test.js
const { add } = require("../math");
describe("Math Operations Suite", () => {
  // Positive Test Case
  test("should correctly add two numbers (Happy Path)", () => {
    {
      // Arrange
      const num1 = 5;
      const num2 = 10;
      // Act
      const result = add(num1, num2);
      // Assert
      expect(result).toBe(15);
    });
    // Negative Test Case
    test("should throw an error if inputs are strings", () => {
      // Act & Assert combined for error handling testing
      expect(() => add("5", 10)).toThrow("Only numbers are
allowed");
    });
  });
});
```

10. Run the Tests, Execute the script in your terminal

npm test

11. Jest will output a structured report showing your test suites, individual test statuses (green checkmarks or red errors), and execution times.

12. **Pass Result:** Terminal highlights the successful test in green showing

5 + 10 = 15 below ↓

```
D:\ngprac\Day43>npm test
> day43@1.0.0 test
> jest

PASS ./math.test.js
Math Operations Suite
  ✓ should correctly add two numbers (Happy Path) (2 ms)
  ✓ should throw an error if inputs are strings (3 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 4.947 s
Ran all test suites.
```

13. **Fail Result:** Terminal highlights the mismatch in red, showing a message like →

Expected: 15

Received: 50.

```
D:\ngprac\Day43>npm test
> day43@1.0.0 test
> jest

FAIL ./math.test.js
Math Operations Suite
  ✗ should correctly add two numbers (Happy Path) (4 ms)
  ✓ should throw an error if inputs are strings (2 ms)

• Math Operations Suite › should correctly add two numbers (Happy Path)

expect(received).toBe(expected) // Object.is equality

Expected: 15
Received: 50

10 |     const result = add(num1, num2);
11 |     // Assert
12 |     expect(result).toBe(15);
    |                     ^
13 |   });
14 |   // Negative Test Case
15 |   test("should throw an error if inputs are strings", () => {

at Object.toBe (math.test.js:12:20)

Test Suites: 1 failed, 1 total
Tests: 1 failed, 1 passed, 2 total
Snapshots: 0 total
Time: 0.397 s, estimated 3 s
Ran all test suites.
```



Angular – 21.X.X

Productivity Meets Scalability



Testing in Angular

Angular testing relies on a robust ecosystem built right into the framework via the Angular CLI. It allows you to systematically verify individual pieces of logic, component DOM interactions, and full application flows to ensure your application remains bug-free and maintainable. Testing in Angular is traditionally categorized into three distinct layers, balancing execution speed against realistic environment simulation:

Test Type	Target Focus	How It Works
Unit Tests	TypeScript Classes (Services, Pipes, Utilities)	Tests methods and asserts against properties in isolation. All dependencies are mocked. Does not interact with the DOM.
Component / Integration Tests	Component + DOM Template	Interacts with the component via the DOM (e.g., clicking buttons, verifying rendering). Uses TestBed to create a sandbox.
End-to-End (E2E) Tests	Entire Application Flow	Runs completely outside the Angular instance. Simulates real user behavior in a browser wrapper using tools like Cypress.

Three Testing Technologies — Big Picture

There are three tools as **three different levels of testing** rather than as competing tools:

- **Jasmine + Karma** → traditional Angular unit testing; important for understanding and maintaining legacy Angular applications.
- **Vitest** → modern, fast unit/integration testing approach; a good choice for new Angular projects.
- **Playwright** → end-to-end (E2E) testing; tests the application through browser like a real user.

Technology	What it tests	Runs where?	Typical use
Jasmine	Unit tests	Test environment	Assertions/specs
Karma	Test runner/browser launcher	Browser	Traditional Angular testing
Vitest	Unit/integration tests	Node/browser-like environment	Modern fast testing
Playwright	E2E tests	Real browsers	Complete user workflows

A very important point here to consider:

Jasmine and Karma are not the same thing.

Jasmine provides the **testing framework and syntax**, while Karma is the **test runner** traditionally used to execute Angular tests in a browser.

Similarly:

Vitest is both a testing framework and test runner.

And:

Playwright is primarily used for browser automation and end-to-end testing.



Angular – 21.X.X

Productivity Meets Scalability



Core Testing Tooling

1. Test Runners & Frameworks

- **Jasmine & Karma:** Historically the defaults, Jasmine provides the BDD (Behavior-Driven Development) syntax (describe, it, expect), while Karma serves as the browser-based test runner.
- **Vitest & jsdom:** Modern versions of Angular use Vitest as the default fast, lightweight test runner, paired with jsdom for simulating the browser environment.
- **Command Execution:** Simply running `ng test` builds your application in watch mode and kicks off your testing suite.

2. Built-in Angular Utilities

- **TestBed:** The primary Angular testing utility. It configures a mock Angular NgModule environment where you declare components and inject mock providers.
- **ComponentFixture:** A handle surrounding a created component that allows you to trigger change detection (`fixture.detectChanges()`) and query the DOM elements.

Modern Testing Enhancements

Recent Angular updates have drastically simplified component testing syntax:

- **Lean Binding Inputs:** Modern versions (Angular 20.1+) permit passing input bindings directly inside the `TestBed.createComponent` argument. This lets you test inputs—including dynamic Signals—without needing to write clunky wrapper components.
- **Third-Party Libraries:** Tools like Testing Angular with Spectator and ng-mocks cut down boilerplate setup code for TestBed, making isolation tests much faster to write.

Best Practices for Writing Angular Specs

- **Keep focus singular:** Dedicate each `it()` block to verifying one specific behavior or outcome.
- **Mock external dependencies:** Use Jasmine/Vitest spies or custom mock classes for services and HTTP requests so your tests don't break due to external server failures.
- **Handle Asynchronicity properly:** Utilize Angular's `fakeAsync + tick()` or `async/await` utilities when testing operations like timeouts, promises, or Observables.
- **Test behaviors, not internals:** Verify what the user sees or what a function returns rather than asserting against private component properties.

Testing with Karma and Jasmine

While Vitest is the default test runner for new Angular projects, Karma is still a supported and widely used test runner. This guide provides instructions for testing your Angular application using the Karma test runner with the Jasmine testing framework.



Angular – 21.X.X

Productivity Meets Scalability



Essential Testing Vocabulary

To get started with unit testing in Angular, it helps to understand the roles of the two main tools: **Karma** acts as your test runner (the engine that opens the browser and runs the suite), while **Jasmine** provides the testing vocabulary (functions like describe, it, and expect) to write the actual checks.

Jasmine is an open-source, behavior-driven development (BDD) testing framework for JavaScript code. It is heavily used for **unit testing** because it is fast, has zero external dependencies ("batteries included"), and features a clean, natural-language syntax. Jasmine is famously integrated natively into **Angular** projects, though it can test any browser or Node.js application.

Test Suite: A Test Suite is a logical collection or group of related test cases. Think of it as a labeled folder that keeps organized all the individual tests written for a specific feature, service, or component. In Angular testing with Jasmine, a test suite is created using the describe() function.

Understand with an Example, if you are testing a car:

- The Test Suite is: "Car Braking System"
- The Individual Test Cases inside it are:
 - "Should stop within 3 seconds when hitting brakes at 50 km/h"
 - "Should engage anti-lock braking (ABS) on wet roads"
 - "Should light up the brake dashboard indicator"

The code below demonstrates how to structure the suite using describe, write individual test cases with it, and set up assertions using mock data.

```
// 1. The TEST SUITE (The folder grouping everything together)
describe('Car Braking System', () => {
  let mockCar: any;
  // This runs before EACH individual test case to reset our car's state
  beforeEach(() => {
    mockCar = {
      speed: 50,
      surfaceCondition: 'dry',
      brakeApplied: false,
      absEngaged: false,
      dashboardIndicatorLit: false,

      // Simulates hitting the brakes
      pressBrakePedal() {
        this.brakeApplied = true;
        if (this.surfaceCondition === 'wet') {
          this.absEngaged = true;
        }
        this.dashboardIndicatorLit = true;
      },

      // Simulates checking how long it takes to full stop
      getSecondsToStop() {
        return this.speed === 50 ? 2.5 : 4;
      }
    };
  });
});
```



Angular – 21.X.X

Productivity Meets Scalability



```
});
});

// 2. INDIVIDUAL TEST CASE 1
it('should stop within 3 seconds when hitting brakes at 50 km/h', () =>
{
  // Arrange
  mockCar.speed = 50;

  // Act
  mockCar.pressBrakePedal();
  const timeSpent = mockCar.getSecondsToStop();

  // Assert (2.5 seconds is less than or equal to 3)
  expect(timeSpent).toBeLessThanOrEqual(3);
});

// 3. INDIVIDUAL TEST CASE 2
it('should engage anti-lock braking (ABS) on wet roads', () => {
  // Arrange
  mockCar.surfaceCondition = 'wet';

  // Act
  mockCar.pressBrakePedal();

  // Assert
  expect(mockCar.absEngaged).toBe(true);
});

// 4. INDIVIDUAL TEST CASE 3
it('should light up the brake dashboard indicator', () => {
  // Act
  mockCar.pressBrakePedal();

  // Assert
  expect(mockCar.dashboardIndicatorLit).toBe(true);
});
});
```

Jasmine uses a highly readable structure to group and execute tests. When looking at a .spec.ts test file, you will routinely see following foundational blocks:

- **TestBed:** The primary Angular testing utility. Think of it as a sandbox that mimics an Angular module, allowing you to configure components, inject services, and simulate runtime environments.
- **describe(string, callback):** A **Test Suite**. It groups together related test cases. You typically name it after the class or component you are testing.
- **it(string, callback):** A **Test Case**. This defines a single, specific capability or behavior you want to verify (e.g., “should add two numbers correctly”).



Angular – 21.X.X



Productivity Meets Scalability

- **beforeEach(callback):** A setup hook. It runs before *every single* it block inside its suite. Use this to reset variables and create fresh instances so your tests do not interfere with each other.
- **expect(actual):** The starting point of an **Assertion**. You pass the real result of your code into it.
- **Matcher (.toBe(), .toEqual(), .toBeTruthy()):** The chaining command that finishes the assertion. It checks if the value in expect() matches your desired outcome.

Setting Up Karma and Jasmine

You can set up Karma and Jasmine for a new project or add it to an existing one.

- **For New Projects:** To create a new project with Karma and Jasmine pre-configured, run the ng new command with the --test-runner=karma option:

```
ng new my-karma-app --test-runner=karma
```

- **For Existing Projects:** To add Karma and Jasmine to an existing project, follow these steps:

- Install the necessary packages:

```
npm install --save-dev karma
```

```
karma-chrome-launcher karma-coverage karma-jasmine
```

```
karma-jasmine-html-reporter jasmine-core @types/jasmine
```

- Configure the test runner in angular.json: In your angular.json file, find the test target and set the runner option to karma:

```
"test": {  
  "builder": "@angular/build:unit-test",  
  "options": {  
    "runner": "karma"  
  }  
}
```

- **Update tsconfig.spec.json for Jasmine types:** To ensure TypeScript recognizes global testing functions like describe and it, add "jasmine" to the types array in your tsconfig.spec.json:

```
{  
  "extends": "./tsconfig.json",  
  "compilerOptions": {  
    "types": ["jasmine"]  
  },  
  "include": [  
    "src/**/*.d.ts",  
    "src/**/*.spec.ts"  
  ]  
}
```



Angular – 21.X.X

Productivity Meets Scalability



Once your project is configured

Add a service to angular application or use any existing one.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class CalculatorService {
  add(a: number, b: number): number {
    return a + b;
  }

  subtract(a: number, b: number): number {
    return a - b;
  }

  multiply(a: number, b: number): number {
    return a * b;
  }

  divide(a: number, b: number): number {
    if (b === 0) {
      throw new Error('Cannot divide by zero');
    }
    return a / b;
  }
}
```

Now modify .spec.ts file to define Test-Suite, check respective changes

```
import { TestBed } from '@angular/core/testing';

import { CalculatorService } from './calculator-service';

describe('CalculatorService', () => {
  let service: CalculatorService;

  beforeEach(() => {
```



Angular – 21.X.X

Productivity Meets Scalability



```
TestBed.configureTestingModule({});
service = TestBed.inject(CalculatorService);
});

it('should be created', () => {
  expect(service).toBeTruthy();
});

it('should add two numbers correctly', () => {
  const result = service.add(2, 3);
  expect(result).toBe(5);
});

it('should subtract two numbers correctly', () => {
  const result = service.subtract(5, 3);
  expect(result).toBe(2);
});

it('should multiply two numbers correctly', () => {
  const result = service.multiply(2, 3);
  expect(result).toBe(6);
});

it('should divide two numbers correctly', () => {
  const result = service.divide(6, 3);
  expect(result).toBe(2);
});

it('should throw an error when dividing by zero', () => {
  expect(() => service.divide(6, 0)).toThrowError('Cannot divide
by zero');
});
});
```

run the tests using the ng test command in terminal:

ng test

The ng test command builds the application in watch mode and launches the Karma test runner.



Angular – 21.X.X

Productivity Meets Scalability



The console output looks like below:

```
D:\ngprac\Day43\my-karma-app>ng test
Initial chunk files      Names      Raw size
chunk-3Z7ONWOG.js      -          2.00 MB
spec-app-app.spec.js   spec-app-app.spec  238.44 kB
chunk-5CGALJCZ.js      -          217.56 kB
test_main.js           test_main  20.98 kB
spec-app-services-calculator-service.spec.js spec-app-services-calculator-service.spec  2.06 kB
polyfills.js           polyfills  749 bytes
jasmine-cleanup-0.js   jasmine-cleanup-0  397 bytes
styles.css             styles     95 bytes

Initial total          2.48 MB

Application bundle generation complete. [1.991 seconds] - 2026-09-14T08:56:17.567Z

Watch mode enabled. Watching for file changes...
14 09 2026 14:26:17.849:WARN [karma]: No captured browser, open http://localhost:9876/
14 09 2026 14:26:17.877:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
14 09 2026 14:26:17.877:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
14 09 2026 14:26:17.891:INFO [launcher]: Starting browser Chrome
14 09 2026 14:26:19.451:INFO [Chrome 152.0.0.0 (Windows 10)]: Connected on socket TyU10SGolPX6Mgh8AAAA with id 31295174
Chrome 152.0.0.0 (Windows 10): Executed 8 of 8 SUCCESS (0.048 secs / 0.034 secs)
TOTAL: 8 SUCCESS
```

The test output is displayed in the browser using Karma Jasmine HTML Reporter.

Here Passed Tests in green and failed one in red and when you click any of that will open its description:

Karma

localhost:9876/?id=31295174

Chrome is being controlled by automated test software

Karma v 6.4.4 - connected; test: complete;

Chrome 152.0.0.0 (Windows 10) is idle

Jasmine 4.6.1

8 specs, 0 failures, randomized with seed 05252

App

- should create the app (17ms)
- should render title (10ms)

CalculatorService

- should be created (2ms)
- should multiply two numbers correctly (1ms)
- should divide two numbers correctly (1ms)
- should throw an error when dividing by zero (2ms)
- should add two numbers correctly (1ms)
- should subtract two numbers correctly (1ms)



Angular – 21.X.X

Productivity Meets Scalability



Karma v 6.4.4 - connected; test: complete;

Chrome 152.0.0.0 (Windows 10) is idle

Jasmine 4.6.1

8 specs, 1 failure, randomized with seed 67157

[Spec List](#) | [Failures](#)

CalculatorService

- ✗ should throw an error when dividing by zero (6ms)
- should multiply two numbers correctly (4ms)
- should subtract two numbers correctly (2ms)
- should add two numbers correctly (2ms)
- should divide two numbers correctly (2ms)
- should be created (2ms)

App

- should create the app (10ms)
- should render title (8ms)

Clicked on Green

Karma v 6.4.4 - connected; test: complete;

Chrome 152.0.0.0 (Windows 10) is idle

Jasmine 4.6.1

Ran 1 of 8 specs - run all

1 spec, 0 failures, randomized with seed 97818

CalculatorService

- should divide two numbers correctly (3ms)

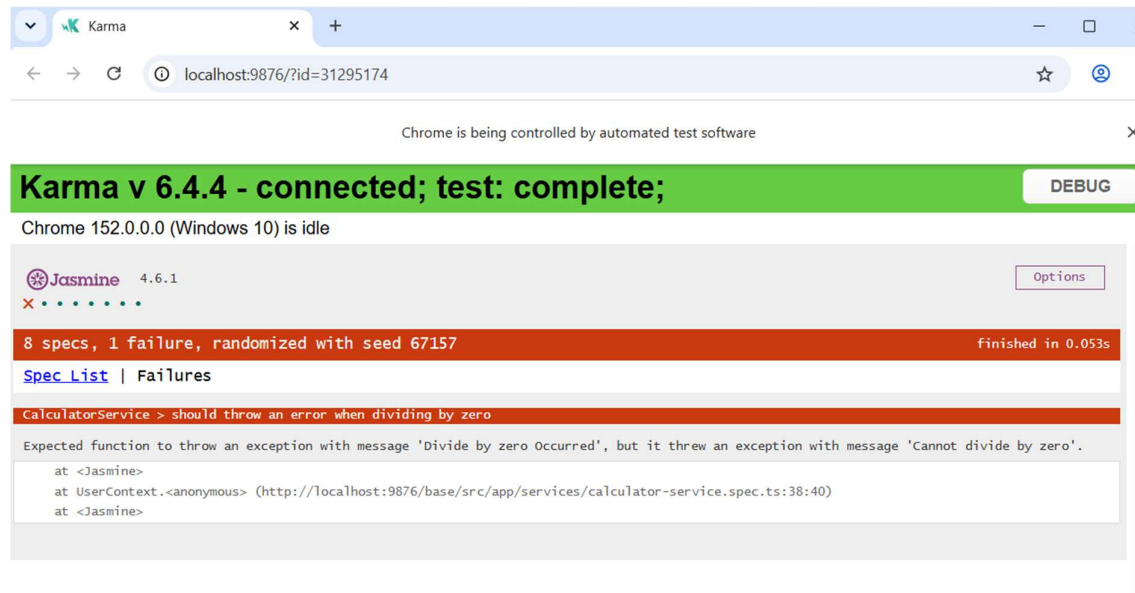


Angular – 21.X.X

Productivity Meets Scalability



Clicked or Red in the specs list:



Click on a test row to re-run just that test or click on a description to re-run the tests in the selected test group ("test suite").

Meanwhile, the ng test command is watching for changes. To see this in action, make a small change to a source file and save. The tests run again, the browser refreshes, and the new test results appear.

Testing in Angular Using Default VITEST Testing

Vitest (pronounced as "veetest") is a next generation testing framework powered by Vite. A Vite-native testing framework is a fast, lightweight unit test framework powered by Vite with a Jest-compatible API.

Vitest is a modern, blistering-fast, Vite-native testing framework that is the official default unit testing tool for Angular projects. Angular officially adopted Vitest to replace the legacy Karma and Jasmine tool stack, providing an immediate performance boost and eliminating the overhead of heavy browser launches during rapid development.

Key Features

- **Vite Integration:** Reuses your Vite config, resolvers, and plugins.
- **Fast Watch Mode:** Reruns only related changes, similar to Hot Module Replacement (HMR).
- **Out-of-the-box Support:** Native ESM, TypeScript, and JSX support.
- **Jest Compatible:** Uses compatible APIs for assertions, snapshots, and mocking.

Core Uses & Benefits of Vitest in Angular

- **Near-Instant Execution:** Leverages Vite's Hot Module Replacement (HMR) to run tests in milliseconds.



Angular – 21.X.X



Productivity Meets Scalability

- **No Mandatory Browser Overhead:** Runs tests inside a fast, emulated Node environment (jsdom or happy-dom) by default, though it can still spin up real browsers via Playwright for deep integration checks.
- **Familiar API:** Uses an API heavily inspired by Jest and Jasmine (describe, it, expect), ensuring a virtually nonexistent learning curve for experienced Angular developers.
- **Native TypeScript Support:** Out-of-the-box support for TypeScript and modern Angular primitives like Signals and zoneless rendering.

... to be continued

⚙️ ❌ Catch the bugs before they change your Signals. ❌ ⚙️